

Avaliação 2 (Trabalho) de Ciência de Dados - NES

prof. Eduardo Adame

05 de junho de 2026

Objetivo da Avaliação

A AV2 é uma **evolução direta do projeto da AV1**. O aluno deverá aprimorar a aplicação já construída, introduzindo três novos componentes: uma **API intermediária** que desacopla o dashboard do banco de dados, um **modelo preditivo** integrado ao dashboard, e uma **análise explícita do processo de escolha e ajuste do modelo**. O objetivo é demonstrar que o projeto passou de um painel descritivo para um sistema de suporte à decisão com capacidade preditiva.

Formato da Apresentação

- **Duração:** 12 minutos por apresentação (10 min demonstração + 2 min perguntas)
- **Data:** 19 de junho de 2026
- **Componentes obrigatórios:**
 - Slides atualizados com a nova arquitetura e a justificativa do modelo escolhido
 - Aplicação completa rodando via **Docker** na máquina local
 - Demonstração ao vivo da API, do dashboard preditivo e da interface de controle do modelo
 - Código-fonte atualizado no mesmo repositório Git da AV1

Cronograma

Data	Atividade
05/06/2026	Lançamento das instruções da AV2
12/06/2026	Prazo final para envio do link do repositório atualizado
19/06/2026	Apresentações dos seminários

Atenção: O repositório deve estar público e o link enviado pelo mesmo canal da AV1 até 12/06/2026. Atrasos resultarão em penalização conforme o Critério 6.

O que deve ser adicionado ao projeto da AV1

1. API intermediária

O dashboard **não pode mais consultar o banco de dados diretamente**. Toda leitura de dados deve passar por uma API construída com **FastAPI**, que expõe ao menos as seguintes rotas:

Método	Rota	Descrição
GET	/dados	Retorna os registros do banco, com filtros por período e categoria via query params
GET	/resumo	Retorna estatísticas agregadas (médias, totais, contagens) por grupo
GET	/health	Health check — retorna {"status": "ok"}

O dashboard passa a ser um **cliente HTTP** da API. Use `httpx` ou `requests` para realizar as chamadas. Ambos os serviços (API e dashboard) devem ser orquestrados pelo `docker-compose.yml`.

2. Modelo preditivo no dashboard

Implemente ao menos **um modelo preditivo** integrado ao dashboard. O modelo deve prever uma variável relevante do seu domínio — exemplos por tópico:

Tópico da AV1	Variável a prever (sugestão)
Desempenho escolar	Nota final de um aluno dado seu histórico parcial
Vendas de e-commerce	Faturamento dos próximos N dias
Dados climáticos	Temperatura ou precipitação nas próximas horas
Saúde pública	Número de atendimentos nos próximos dias
Mercado financeiro	Preço ou retorno do ativo no próximo período
Repositórios GitHub	Crescimento de estrelas nos próximos dias

A **interface do dashboard** deve permitir ao usuário:

- Selecionar a variável a prever e o horizonte de previsão
- Escolher entre ao menos **dois modelos diferentes** (ex: regressão linear vs. Random Forest; ARIMA vs. Prophet)

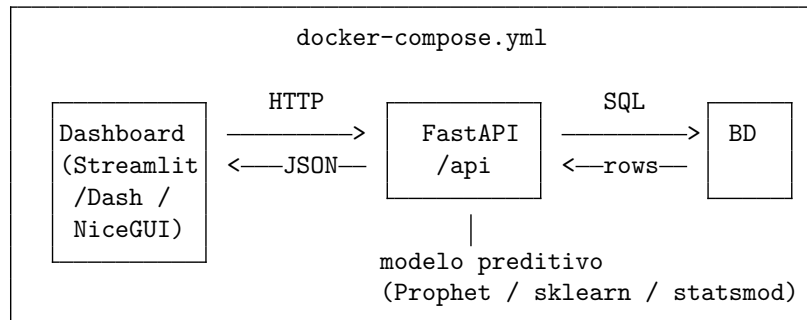
- Definir o **corte temporal t0**: dados até t0 são treino, dados após t0 são usados para comparação com a previsão
- Visualizar a previsão como linha tracejada sobreposta à série real, com intervalo de confiança quando disponível

3. Análise do modelo nos slides

Os slides devem incluir uma seção dedicada ao modelo, cobrindo:

- **Motivação**: por que esse modelo faz sentido para o domínio?
- **Pré-processamento**: quais transformações foram necessárias nos dados?
- **Escolha de hiperparâmetros**: como foram selecionados? (grid search, cross-validation, inspeção manual)
- **Métricas de avaliação**: ao menos duas métricas reportadas no período de teste (ex: MAE, RMSE, MAPE, F1)
- **Limitações**: o que o modelo não consegue capturar? Em que condições ele falha?

Arquitetura mínima obrigatória



O projeto deve continuar rodando do zero com:

```
docker compose up
```

Critérios de Avaliação

A avaliação é composta por **5 critérios**, totalizando **10,0 pontos**, com possibilidade de **+1,0 ponto extra**:

1. Demonstração ao vivo (2,5 pontos)

- **2,5 pontos**: API respondendo, dashboard preditivo funcionando, controles interativos operacionais, tudo via Docker
- **1,5 pontos**: Dashboard funcional mas API com problemas, ou sem Docker
- **0,5 pontos**: Aplicação parcialmente funcional ou demonstrada apenas em vídeo

- **0,0 pontos:** Projeto não roda ou não é demonstrado

2. API intermediária (2,0 pontos)

- Rotas `/dados`, `/resumo` e `/health` implementadas e funcionais
- Filtros via query params operacionais (período, categoria ou cidade)
- Dashboard consumindo a API via HTTP — nenhuma query SQL direta no código do dashboard
- Documentação automática acessível em `/docs` (gerada automaticamente pelo FastAPI)

3. Modelo preditivo (2,5 pontos)

- Ao menos dois modelos implementados e selecionáveis na interface
- Corte t_0 configurável pelo usuário via seletor no dashboard
- Visualização correta: série real, previsão (linha tracejada) e IC quando disponível
- Métricas de avaliação (MAE + RMSE no mínimo) exibidas no dashboard para o período de teste

4. Análise do modelo nos slides (2 pontos)

- Motivação e escolha do modelo justificadas com argumentos técnicos
- Processo de ajuste de hiperparâmetros documentado (não basta dizer “usamos os valores padrão”)
- Métricas reportadas com interpretação — o que significa um MAE de X para esse domínio?
- Limitações do modelo discutidas honestamente

5. Qualidade do código e repositório (1,0 ponto)

- Estrutura de pastas clara separando `api/`, `dashboard/` e `db/` (ou equivalente)
- `docker-compose.yml` com pelo menos dois serviços bem definidos
- `README.md` atualizado com instruções da AV2
- Histórico de commits mostrando evolução desde a AV1 (não commitar tudo de uma vez)

Bônus: - **+0,5 pontos** se os slides forem produzidos em **LaTeX / Beamer** ou **Typst** - **+0,5 pontos** se a aplicação estiver em **deploy em produção** acessível publicamente (Railway, Render, Fly.io, VPS ou similar) com URL funcional no `README.md`

Entregáveis

No dia da apresentação, o aluno deverá:

1. Chegar com o **projeto rodando** (`docker compose up` já executado)

2. O repositório Git deve conter, além do que já existia na AV1:
 - `api/` com o código FastAPI
 - `docker-compose.yml` atualizado com o serviço da API
 - `README.md` atualizado documentando as rotas da API e as instruções da AV2
 - Slides em PDF com a seção de análise do modelo

Não é necessário entregar relatório escrito — os slides e a demonstração ao vivo substituem essa etapa.

Relação com a AV1

A AV2 **não substitui** a AV1 — as notas são independentes. O projeto é o mesmo, mas as adições da AV2 serão avaliadas por si mesmas. Um projeto que não funcionou na AV1 pode funcionar na AV2 e receber nota plena nesta avaliação.

O ponto de partida esperado é o projeto entregue na AV1. Caso o aluno queira mudar de tópico, deve comunicar ao professor até **08/06/2026**.

Dicas

- Separe a lógica de previsão em um arquivo próprio (`modelo.py` ou similar) — não misture código de ML com código de layout do dashboard
- Prophet funciona bem para séries temporais com sazonalidade e requer poucos dados; scikit-learn é mais flexível para dados tabulares — escolha de acordo com o seu domínio
- O FastAPI gera documentação interativa automaticamente em `http://localhost:8000/docs` — use isso na apresentação para mostrar as rotas
- Teste o `docker compose up` em um ambiente limpo antes da apresentação
- Para o deploy em produção (bônus), Railway e Render têm planos gratuitos que suportam containers Docker sem configuração complexa